

TP2: Détection non supervisée d'anomalies

02 Mars 2026

Dans ce deuxième TP, nous étudierons la détection d'anomalies (non supervisée). Le langage de programmation utilisé est toujours **Python**. Vous utiliserez toujours l'environnement virtuel *TP_VD* du TP1. Pour toutes les manipulations, n'hésitez pas à consulter la documentation des méthodes et bibliothèques concernées. L'utilisation d'un notebook est fortement encouragée.

Génération d'un jeu de données synthétique

Nous allons commencer par utiliser un jeu de données synthétiques. Pour ce faire, il sera généré.

1. En utilisant la fonction *make_blobs* de *sklearn.datasets* (https://scikit-learn.org/stable/modules/generated/sklearn.datasets.make_blobs.html), générer un jeu de données contenant **un** cluster de **300** données en deux dimensions. Les données du cluster ont un écart-type (hint: *standard deviation*) de 0.50.
2. Rajouter **trente** anomalies au jeu de données ainsi généré. Utilisez la fonction *uniform* de *numpy.random* (<https://numpy.org/doc/2.2/reference/random/generated/numpy.random.uniform.html>). La bibliothèque **numpy** est installée avec **scikit-learn**, donc vous devez juste l'importer. Vous pouvez lui donner un alias au moment de l'importation. L'alias le plus utilisé est *np*.
3. Créer le jeu de données final : il contient le cluster de données régulières et les anomalies. Stockez-le dans une variable *X*. (Rappel : *X* est une matrice en deux dimensions. La première dimension de la matrice contient chaque point x_i du jeu de données et la deuxième dimension contient les dimensions de chaque point x_i . Par conséquent, *X* a n lignes et D colonnes. Rappelez-vous du jeu de données *digits* du TP précédent.) Comme dit en cours, nous aurons besoin des étiquettes lors de l'évaluation. Donc gardez aussi les étiquettes de chaque point dans un vecteur *y*. Les anomalies ont pour étiquettes -1 et les données régulières ont pour étiquettes 1 .
4. Afficher le jeu de données en utilisant **matplotlib**. Ajuster le jeu de données au besoin : lors de l'ajout des anomalies avec la fonction *np.random.uniform()*, ces anomalies peuvent se retrouver dans le cluster de données régulières et devront donc être supprimées du jeu de données (ou alors vous changez simplement l'étiquette du point concerné).

Forêts d'isolation

Dans cette section, nous appliquons les forêts d'isolation (<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html>) sur notre jeu de données généré précédemment.

1. Construire une forêt d'isolation sur les données. Utilisez les valeurs par défaut des hyperparamètres. Afficher les résultats : choisir une couleur pour afficher les anomalies et une autre pour afficher les données régulières.
2. Exécuter le code de la question 1 plusieurs fois. Que remarquez-vous à l'affichage ?
3. Construire une forêt d'isolation sur les données comme à la question 1 mais, cette fois, donner une valeur au paramètre *random_state*. Les valeurs des hyperparamètres sont les valeurs par défaut. Afficher les résultats. Relancer cette portion de code plusieurs fois. Que remarquez-vous ?
4. Construire une forêt d'isolation sur les données. Fixer le random state. Faire varier le paramètre *contamination*. Afficher les résultats. Que remarquez-vous ?

En pratique, vous ne saurez pas a priori combien d'anomalies il y a dans le jeu de données. Vous obtiendrez des scores d'anomalies.

5. Construire une forêt d'isolation (valeurs par défaut des hyperparamètres, mais fixer le random state). Calculer les scores d'anomalies des points du jeu de données. Observer ces scores. Êtes-vous en mesure de choisir un seuil d'anomalies ?
6. Afficher le jeu de données en changeant l'opacité de chaque point en fonction de son score d'anomalies. Que remarquez-vous ?
7. Faire varier le seuil d'anomalies et afficher les résultats : une couleur pour afficher les anomalies et une autre pour afficher les données régulières.
8. Construire une forêt d'isolation avec les valeurs par défaut des hyper-paramètres (fixer le random state) et une *contamination* égale à la vraie contamination du jeu de données, i.e. 0.09 (9%).
9. Calculer l'AUC de la forêt de la question 5. Utiliser à cet effet la fonction *roc_auc_score* de *sklearn.metrics* (https://scikit-learn.org/stable/modules/generated/sklearn.metrics.roc_auc_score.html).

One-class SVMs

Dans cette section, nous appliquons les One-Class SVMs (<https://scikit-learn.org/stable/modules/generated/sklearn.svm.OneClassSVM.html>) sur notre jeu de données.

1. Normaliser le jeu de données en utilisant le *Standard Scaler* de *sklearn.preprocessing* (<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html>). Nous verrons la normalisation en détails dans un prochain TP.
2. Appliquer les One-Class SVMs sur le jeu de données normalisé avec les valeurs par défaut des hyper-paramètres. Afficher les résultats : choisir une couleur pour afficher les anomalies et une autre pour afficher les données régulières. Comment trouvez-vous les résultats ?
3. Appliquer les OCSVMs avec différentes valeurs de l'hyperparamètre ν (nu). Afficher les résultats obtenus pour les différentes valeurs de ν . Que remarquez-vous ?
4. Calculer l'AUC du meilleur résultat de la question précédente.

LOF

Dans cette section, nous appliquons LOF (<https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.LocalOutlierFactor.html>) sur notre jeu de données.

1. Utiliser le jeu de données normalisé de la question 1 de la section précédente. Appliquer LOF avec les valeurs par défaut des hyperparamètres. Afficher les résultats obtenus : choisir une couleur pour afficher les anomalies et une autre pour afficher les données régulières. Comment trouvez-vous les résultats ?
2. Appliquer LOF différentes valeurs pour le nombre de voisins (*n_neighbors*). Aller jusqu'à 300. Afficher les résultats obtenus. Que remarquez-vous ?
3. Appliquer LOF avec la valeur par défaut du nombre de voisins. Afficher les scores d'anomalies de tous les points. Êtes-vous en mesure de choisir un seuil ?
4. Afficher le jeu de données en changeant l'opacité de chaque point en fonction de son score d'anomalies. Que remarquez-vous ?
5. Appliquer LOF avec la valeur par défaut du nombre de voisins et une *contamination* égale à la vraie contamination du jeu de données, i.e. 0.09 (9%).
6. Calculer l'AUC de la question 3.

Jeu de données réel

Dans cette section nous utiliserons le jeu de données réel Annthyroid. Il est disponible sur Moodle. Il possède 7200 instances (ou observations) et 6 dimensions. La dernière colonne contient les étiquettes de chaque point : 0 pour les données régulières et 1 pour les anomalies.

1. Stocker les données dans une matrice X et les étiquettes dans un vecteur y . Vous pouvez utiliser la fonction *loadtxt* de *numpy* (<https://numpy.org/doc/2.2/reference/generated/numpy.loadtxt.html>).
2. Comparer la performance des trois algorithmes sur le jeu de données.